Admission Is a Feasibility Test, Not a Threshold: Capacity Accounting and Revival Pricing for Agentic LLM Serving

Anonymous Submission

***Abstract*—An agentic LLM session is a long, multi-round job. Every round returns with a context that has grown, so a session's KV footprint is large, durable, and reused—and on a single GPU the live working set can exceed the KV pool by an order of magnitude. Existing systems frame this as placement, and placement forces prediction: which prefix will be reused, when a session will come back, how long the next round will be. We argue the prediction is unnecessary. A session can be *suspended* at a round boundary and *revived* by copying its KV back—when the host tier still holds it—instead of recomputing it; admission then becomes a feasibility test over a capacity ledger—does this session's projected footprint fit, now, on which tier, and at what revival cost. We give a feasible-set bound, a session-level revival-versus-recompute break-even, and a coverage-bounded growth estimate that turns the safety margin into a stated property rather than a constant. On a 19-session, 602-round coding-agent replay, the terminal working set is 12.9× the KV pool and a mean session needs 69K of its 101K tokens, so the pool holds 1.47 mean sessions. Replaying every round as an independent request re-reads 13× more prefix than it appends. We instantiate the design on SGLang with a two-tier KV cache and compare it against request-level serving and an agent-level feedback controller: [PENDING: headline number]. We also state where the mechanism loses— when the working set fits, and when revival costs as much as recomputation.**

***Index Terms*—LLM serving, agentic workloads, KV cache, admission control, capacity accounting, performance evaluation**

I. Introduction

A coding agent does not issue one request. It issues a *session*: a multi-round job in which every round ends with a tool call, and every tool call returns with more context than the round before. The session is long-lived, its state is expensive to rebuild, and it is reused. On a serving engine that was designed for independent requests, none of these three properties is visible.

We measured this on a replay of 19 long coding-agent sessions (602 rounds, median 28 rounds per session, longest 81) collected from Claude and Codex conversations. A round arrives with a median 48.5K-token context (p90 71.1K, max 92.8K). The terminal working set—what all 19 sessions would occupy if every one of them were live—is 1.31M tokens. The KV pool of our engine, a 7B model served with a 96K context on a single 24 GB GPU, holds 101.5K tokens. The working set is therefore 12.9× the pool, and the average session alone needs 69K tokens: *the pool holds 1.47 sessions*.

The cost of ignoring this is paid every round. A round's new content is a median of 990 tokens, but it arrives attached to a 44.9K-token prefix. Serving each round as an independent request re-reads that prefix: across the trace, the engine is asked to process 13.0× more prefix than it appends, and a system with no reuse at all would do 14.0× the prefill work of a perfect one. The trace makes the same point in the other direction: it is cheap to keep a session, and expensive to rebuild it.

The obvious response—evict and recompute—runs into the shape of the workload rather than its size. Tool waits are bimodal: the median gap between rounds is 181 ms, the mean is 15.9 s, and the 13.1% of gaps longer than ten seconds account for 93.4% of all parked time. A session's idle period therefore tells you almost nothing about the next one: half of the time the agent is back before an eviction decision could pay for itself, and the other half it is gone for tens of seconds. A policy that suspends on every gap recomputes constantly; a policy that never suspends holds sessions it cannot afford.

a) Prior fixes treat this as placement, and placement forces prediction: Paged attention, prefix caching, and tiered KV stores decide *where* a block lives; their admission counterpart is a replacement decision that presupposes a guess about the future—which prefix will be reused, when a session will return, how long its next round will be. The systems that address agentic workloads keep that presupposition: TTL-based scheduling predicts a session's reuse window, program-aware admission predicts the next step's demand, feedback controllers adjust a concurrency limit after congestion is observed, and overload handlers shed sessions outright. Each is a threshold on a predicted quantity. None of them states which set of sessions is *feasible* under the capacity it actually has, and none prices what it costs to bring a suspended session back.

b) Our position: make the session suspendable, then stop predicting: If a session can be paused at a round boundary and resumed by *copying* its KV back from a host tier, then the question "should this session be admitted?" stops being a forecast and becomes an accounting check against a ledger of bytes:

Admission is a feasibility test, not a threshold.

Three things follow, and they organise the paper. First, feasibility is stated in quantities we can measure now—the session's projected footprint, the capacity of each tier, the bandwidth between them—so no predictor is on the critical path. Second, suspension is not free and not always right: reviving a session costs a copy, recomputing it costs a prefill, and the two are comparable for short contexts, which yields

a per-session break-even instead of a policy constant. Third, the one genuinely uncertain quantity, how much a session will grow before it returns, can be bounded by coverage rather than guessed by a point estimate, which turns the safety margin into a property one can state.

c) Contributions:

1) **Formulation** (§III): we cast admission for suspendable agent sessions as a feasibility problem over a two-tier capacity ledger, with an explicit revival cost. The formulation replaces a threshold on predicted load with a test on measured capacity, and it exposes the one parameter that remains—the fraction of the pool held in reserve (§IV).
2) **Mechanism** (§IV–VI): named permits make the ledger enforceable at request granularity; reversible suspension at round boundaries keeps a session's state in the host tier instead of discarding it; a coverage-bounded growth estimate replaces a point forecast; and a revival break-even decides, per suspension, whether to copy or to recompute. We implement the design on SGLang with a hierarchical KV cache and a 96K-context 7B model.
3) **Analysis and evidence** (§V, §VII): we give a feasible-set bound, a session-level revival-versus-recompute break-even, and a coverage-bounded admission guarantee; we evaluate request-level serving, an agent-level feedback controller, and our design on the same trace, engine, and memory budget; and we report the regimes in which suspension does not pay.

d) Results in one line: [PENDING: one-sentence headline with the three arms, e.g. "on the 602-round replay our design finishes the batch in X minutes against Y (request-level) and Z (agent-level), while meeting the per-round deadline on W% of rounds against V% and U%"].

e) Scope: Our testbed is a single 24 GB GPU running a 7B model with a 96K context. We choose this point because it makes the regime sharp, not because the mechanism is specific to the card: the binding variable is the ratio of the live working set to KV capacity, and when that ratio is small, admission is not the problem. We do not claim that our constants transfer to larger devices or to clusters; we do claim that the ledger, the break-even, and the coverage bound are stated in quantities—bytes, bandwidth, session horizon—that do transfer (§IX).

II. Background: What a Long Session Costs

A. The replay

Our workload is a replay of 19 coding-agent sessions (602 rounds) reconstructed from real Claude and Codex transcripts released with a public coding-agent characterization study [1]. We keep every session with at least 20 rounds and sample by a hash of the session identifier, which yields the 19 longest sessions rather than a hand-picked subset; the trace and the script that rebuilds it are in the artifact. Each round carries the prefix it can reuse, the new tokens it appends, the number of tokens the model is asked to emit, an inter-round tool wait, and an arrival time. All sessions arrive at $t = 0$: we model the batch-of-agents case (a nightly evaluation, a benchmark sweep, a fleet of assistants started together), which is the case that stresses admission hardest and the case an offline batch server is actually configured for.

B. Four numbers that decide the design

Table I collects the characterisation; every entry is regenerated from the trace by `gen_trace_facts.py`. Four of them drive everything that follows.

a) The working set is an order of magnitude larger than the pool: The terminal footprint of all 19 sessions is 1.31M tokens. The engine's KV pool on a 24 GB card with a 96K context and a 7B model is 101.5K tokens. The ratio is $12.9\times$, and a mean session (69K tokens) already needs two thirds of the pool: the pool holds 1.47 sessions. There is no scheduling policy that makes 19 sessions resident; the question is only which subset is resident at each instant.

b) Almost all of the work is re-reading: A round appends a median of 990 tokens to a median 44.9K-token prefix. Aggregated over the trace, the prefix is $13.0\times$ the appended content, and the prefill work of a system with no reuse would be $14.0\times$ that of a system that never re-reads a prefix. This gap is the prize: the engine spends its time on prefill of content it has already seen.

c) Reuse is worth a lot, and it is perishable: Because a session's context only grows, the prefix of round i is a prefix of round $i+1$: a single retained copy serves every later round. But retention competes with admission—every token held for a parked session is a token an active session cannot use—so reuse and capacity are two views of the same decision, not two separate optimisations.

d) Idle time is bimodal, so "park" is not one thing: The median gap between rounds is 181 ms; the mean is 15.9 s; the 90th percentile is 28.1 s and the maximum is the trace's 300 s timeout. The 13.1% of gaps above ten seconds carry 93.4% of all parked time: across the trace the sessions sit idle for 2.65 hours in total, and 2.47 of those hours are in long gaps. A gap of 181 ms is shorter than the cost of moving a 44.9K-token prefix anywhere, while a gap of 28 s is longer than the prefill that would rebuild it. Any single policy—always keep, always evict—is wrong for at least half of the gaps.

C. What a round does to the engine

A round is a prefill of the full context followed by a short decode: 48.5K tokens in, about 150 tokens out. On the 24 GB card the prefill of a cold 44.9K-token prefix takes about 100 s (median 103 s over the 131 cold rounds of one run) and the decode is a fraction of a second; when the prefix is resident, the same round costs a few milliseconds of copy and a prefill of the 990 new tokens. The engine therefore has two very different operating points, and the whole question of this paper is how often it can be in the second one.

D. The engine and its memory hierarchy

We use SGLang [2] with paged KV [3] and its hierarchical cache, which extends the device pool with a host pool at

TABLE I: Workload characterisation of the 19-session, 602-round replay. Context is measured at the start of a round; the working set sums the terminal context of each session. All values are recomputed from the trace by `gen_trace_facts.py`.

Quantity	Value
sessions / rounds	19 / 602
rounds per session (median / max)	28 / 81
context per round (median / mean)	48.5K / 48.2K
context per round (p90 / max)	71.1K / 92.8K
terminal working set	1.31M tokens
KV pool (device tier)	101.5K tokens
working set / pool	12.9×
mean session footprint	69.0K tokens
pool in mean sessions	1.47
appended tokens per round (median)	990
reusable prefix per round (median)	44.9K
prefix / append, per round (median)	45.4×
prefix / append, whole trace	13.0×
recompute / append bound	14.0×
tool wait p50 / mean	181 ms / 15.9 s
tool wait p90 / max	28.1 s / 300 s
gaps > 10 s (share of gaps / of time)	13.1% / 93.4%

a configurable ratio (we use 2×, i.e. 203K tokens in host memory). The hierarchy is the substrate that makes suspension meaningful: the host tier is where a suspended session can wait without occupying the card, and the PCIe link is the price of putting it back. The engine itself does not have a notion of a session, so it cannot tell a cache miss caused by a new session from one caused by an evicted session that is about to return. Supplying that notion is the job of the design in §IV.

III. Model

A. Sessions, rounds, and state

A *session* s is a sequence of rounds $i = 1, \dots, n_s$, arriving at time a_s. Round i carries a reusable prefix of $p_{s,i}$ tokens, appends $q_{s,i}$ tokens, and asks for $o_{s,i}$ output tokens, so the context served by the engine is $c_{s,i} = p_{s,i} + q_{s,i}$ and the KV state after the round occupies $\Phi_{s,i} = \kappa\,(c_{s,i} + o_{s,i})$ bytes for κ bytes per token. Between rounds the session is idle for $w_{s,i}$ (the tool call), which in our trace is drawn from a bimodal distribution (§II). Because a session only appends, the prefix of round i is a prefix of every later round: the reuse unit is the session, not the request.

At any instant a live session is in one of three *states*: *resident* (its KV is in the device pool, so its next round prefills only $q_{s,i}$), *suspended* (its KV is in the host tier, so its next round costs a copy plus $q_{s,i}$), or *absent* (its state is gone, so its next round prefills all of $c_{s,i}$). The engine we build on implements the first and third; the design in §IV adds the second.

B. Capacity and the feasible set

Let the device tier hold C_d tokens and the host tier C_h tokens, and let $\rho \in (0, 1]$ be the fraction of the device pool the admission policy is allowed to commit. At time t let $A(t)$ be the resident set and $H(t)$ the suspended set. Feasibility is

$$\sum_{s \in A(t)} \Phi_s(t) \;\le\; \rho\, C_d, \qquad \sum_{s \in H(t)} \Phi_s(t) \;\le\; C_h. \tag{1}$$

The *feasible set* $\mathcal{F}(t)$ is the collection of resident sets satisfying (1); admission is the choice of $A(t)$, not of a concurrency count. This is the formal content of our claim. A threshold policy fixes $|A(t)| \le k$; because Φ_s varies by 2× across sessions and by 2× within a session over its life, a fixed k is either infeasible for some t or leaves capacity unused at others. The bound in §V makes the gap explicit.

C. Two costs: revive or recompute

Bringing a session back has two possible prices. Copying it back from the host tier costs

$$T_{\mathrm{rev}}(s) \;=\; \frac{\Phi_s}{B} \;+\; \tau, \tag{2}$$

where B is the effective host-to-device bandwidth and τ the fixed cost of restoring a sequence in the engine (allocation, descriptor rebuild, first-token latency). Recomputing the round from scratch costs the prefill of the whole context,

$$T_{\mathrm{rec}}(s) \;=\; P(c_s)\,, \tag{3}$$

with P the prefill latency of the engine at the current batch load. For our workload the two are within a factor of a few: a 45K-token prefill on the 24 GB card takes about 100 s, while copying the corresponding 2.5 GB of KV at [PENDING: GB/s] takes [PENDING: copy time]. Neither dominates, which is why the decision has to be priced rather than hard-coded.

D. Assumptions

We state the assumptions that the analysis uses, so that the guarantees are not read more broadly than they hold.

1) **Round-boundary suspension.** A session is suspended only between rounds. A round in flight is never preempted; this matches the engine, whose KV for an active sequence cannot be moved mid-decode.
2) **Monotone growth.** Within a session the context grows: $c_{s,i+1} \ge c_{s,i}$. This holds in our trace and in the agent workloads we are aware of, and it is what makes a retained prefix reusable.
3) **Known per-token cost.** κ, B, and τ are measured once on the deployment and are stable across sessions. We do not model bandwidth contention between concurrent copies.
4) **Host capacity is finite and shared.** C_h is a real budget; suspending always has a victim when the host is full. This is what makes the break-even a *replacement* condition rather than a per-session one.
5) **Growth is estimated with coverage, not point-predicted.** The estimator returns $\hat{g}_s$ such that the next round's growth $q_{s,i+1} + o_{s,i+1}$ is at most $\hat{g}_s$ with probability at least $1 - \delta$.

IV. Design

The design has one idea and four mechanisms. The idea is that the controller should keep a byte-accurate account of which session occupies which tier, and should make every admission decision against that account. The mechanisms are what it takes to keep the account true: a way to *enforce* it (§IV-A), a way to *hold* a session that is not being served (§IV-B), a way to *reserve* room for growth (§IV-C), and a way to *price* the two decisions the account cannot make by itself (§IV-D). Figure 1 shows the control path.

A. M1: a capacity ledger with named permits

The controller keeps one entry per live session, $\ell_s = (\mathrm{id}, \mathrm{tier}, \hat{\Phi}_s)$, where $\hat{\Phi}_s$ is the projected footprint the session needs for its next round. The projection has two parts: the session's *measured* resident context—the prompt length of its completed rounds, as reported by the engine—and a growth reserve $\hat{g}_s$ (§IV-C). The ledger maintains $\sum_{s\in A} \hat{\Phi}_s$ and $\sum_{s\in H} \hat{\Phi}_s$ and checks them against (1) before every decision.

Accounting alone is only a model, so the decision is made binding by a *named permit*. Once per control cycle the controller writes the complete set of session ids allowed to hold an engine slot, together with the set that is paused; the runner blocks any session whose id is not in the admit set at its next round boundary. There is no scalar concurrency limit anywhere in the path: the permit names sessions, so the set the ledger has accounted for and the set the engine may serve are the same set by construction. This is what separates the design from a threshold controller—a threshold changes how many requests may run and hopes the resulting set fits, whereas a permit cannot be issued until the set has been shown to fit.

B. M2: suspension at round boundaries, not eviction mid-flight

When a session is not allowed to continue, the design does not let the engine evict it mid-round and does not drop its state. The session is *suspended*: it keeps its ledger entry, its permit identity, and its KV, and it blocks at the next round boundary until the ledger admits it again. Suspension is a state of the session, not a side effect of cache replacement.

Two properties make this work in practice. The engine's hierarchical cache keeps the prefix of an evicted sequence in the host tier, so a suspended session that is admitted again usually resumes by *copying* its prefix back rather than recomputing it (§V shows the copy is two orders of magnitude cheaper than the prefill). And the host tier is finite, so suspension is a replacement decision like any other: admitting a session's state into the host tier means some other session's state cannot be there. The ledger therefore tracks host occupancy as a first-class sum, and §IV-D decides who gets a host slot.

C. M3: reserve for growth from observed growth

A session that is admitted now will keep growing while it is resident, so the ledger must reserve room for the next round. We reserve the session's own recent growth:

$$\hat{g}_s = \underbrace{\mathrm{EMA}\big(q_{s,j} + o_{s,j}\big)}_{\text{own history}} \cdot h$$

over the session's completed rounds, where h is the number of rounds the session may run before the next control decision (three in our runs, with a 1.15 safety factor). For a session with fewer than three observed rounds the estimate is blended toward a global prior estimated from all sessions, so a young session is not over-reserved.

The justification for estimating growth so crudely is in the numbers. On the trace the reserve has a median of 3.7K tokens against a 48.5K-token context—*7.7% of the footprint it protects*—and it covers the next round's growth in 99.5% of the 583 round transitions. Two things follow. The reserve is a statistic of the session's own history, not a model of its future, so it cannot be wrong in the way a TTL or a steps-to-execution predictor can be wrong; and because it is under a tenth of the footprint, even a two-fold error in it moves the ledger by less than 8%, which is smaller than the spread between the mean session (69.0K) and the median one (48.5K). The measured part of the projection—the resident context, read from the engine's own records—is what admission actually turns on.

D. M4: pricing the two decisions the ledger cannot make

The ledger says what fits; it does not say what is worth doing. Two decisions remain, and both are priced in the same currency—time.

a) Who gives way, and when: When the pool is over-committed (the engine's own utilisation crosses the budget, or the observed first-token latency crosses the deadline), the controller suspends sessions until the remainder fits. Two rules order the victims. First, it suspends the *youngest* sessions first: they have the least invested context, so the recompute they may incur is the smallest, and they are the least likely to have accumulated state worth keeping. Second, it suspends only sessions that have been idle for longer than a break-even time t^*: a session that has been parked for 100 ms will return before a replacement round can use the slot, so pausing it buys nothing and risks a recompute. We calibrate t^* from the measured tool-wait distribution—the time to serve one round of another session, [PENDING: t-star]—rather than tuning it.

b) Resuming: A suspended session re-enters the ordinary admission path and is admitted when the ledger's projection fits. Because its prefix is usually still in the host tier, resumption is a copy; when the host tier has lost the prefix the same round is served as a cold prefill, and the accounting charges it as such.

c) The round gate: The controller also caps how many sessions may be *running* (as opposed to live) in the engine. The distinction matters: in our trace a cap on live sessions leaves the engine idle whenever all of the admitted sessions happen to be parked, whereas a cap on the running count keeps the engine's concurrent work at the intended level and lets suspended sessions queue at their boundaries. The gate

Figure placeholder: control path

runner → arrival/pause/resume events → ledger (resident + $\hat{g}_s$) → named permit (admit set, paused set) → runner gate → engine
engine `/metrics` → valve → shedding (youngest, idle $> t^*$)

Fig. 1: Control path. The permit names sessions, not a count; suspension and resumption are ledger transitions enforced at round boundaries.

is driven by the engine's own running-request counter, not by the controller's session count.

E. The knob

One parameter remains: the fraction ρ of the pool the ledger is allowed to commit ($\rho = 0.95$ in our runs, with the fixpoint pushed to 0.90 of the budget before shedding). It is the only tuned quantity in the design. Everything else—the projected footprint, the resident context, the running count, the observed latency—is measured. We report the sensitivity of the evaluation to ρ rather than presenting it as a constant of the mechanism.

F. What is deliberately absent

There is no predictor of the next round's length, no estimate of a session's remaining horizon, and no per-request latency model. The only estimated quantity is a per-session growth reserve whose error cannot move the ledger by more than a few percent, and the only tuned quantity is ρ. The design degrades gracefully at both ends: with ample capacity nothing is suspended and the ledger admits first-come-first-served; with short contexts the growth reserve is negligible and admission is a pure capacity check.

V. Analysis

We state three results. The first bounds the resident set in sessions and shows why a concurrency threshold cannot be the right control variable. The second prices a revival and locates the real scarcity. The third bounds how much a wrong growth reserve can move the ledger, which is the formal reason the design can afford a crude estimate.

A. Prop. 1: the feasible set is small, and a threshold cannot track it

Let $\bar{\Phi}(t)$ be the mean projected footprint of the live sessions and ρC_d the committed device budget. Any resident set satisfying (1) obeys

$$|\mathcal{F}(t)| \;\le\; \frac{\rho\, C_d}{\bar{\Phi}(t)}. \tag{4}$$

At $\rho = 0.95$ our budget is 96.4K tokens against a mean session footprint of 69.0K, so the bound is 1.40: *at most one session is resident*, and a second fits only if it is smaller than the mean. The bound follows from the working-set ratio alone: when the working set is m times the pool, at most $|\mathcal{S}|/m$ sessions are resident, here $19/12.9 = 1.47$.

Three consequences contradict a habit of the literature. (i) A concurrency threshold k is infeasible for some t unless $k \le 1$: with two median sessions, $2 \times 48.5\text{K} = 97.0\text{K}$ exceeds the 96.4K budget, so $k = 2$ sits exactly on the edge and $k = 3$ is always over-committed. The threshold is not a tuning knob but a constant that the workload invalidates. (ii) Queueing reasoning about arrival rates is the wrong instrument: there is no stable operating point with many sessions in flight, so latency is set not by utilisation but by *which* session holds the card. (iii) The interesting quantity is not how many sessions are admitted but how much recomputable work is kept alive per byte held—which is what §IV-D ranks.

B. Prop. 2: revivals are cheap; slots are not

With a linear prefill model $P(c) = c/R$ and per-token KV size κ, substituting (2) and (3) gives

$$\frac{T_{\text{rev}}(s)}{T_{\text{rec}}(s)} = \frac{\kappa R}{B} + \frac{\tau R}{c}. \tag{5}$$

On our deployment $\kappa = 56\,\text{KiB/token}$ (the model's KV shape) and $R = 465\,\text{tok/s}$ (the median over 131 cold rounds), with $B = [PENDING : GB/s]$ measured for host-to-device copies, so the first term is $[PENDING : ratio]$ and *restoring a session's state costs about two orders of magnitude less than recomputing it*, for any context beyond a few hundred tokens.

This changes what the problem is. If revivals were expensive, the right design would be a careful oracle about when to keep state; the measurement says the transfer is not the bottleneck—the *slot* is. The host tier holds $C_h = 203\text{K}$ tokens, and the working set is $12.9\times$ the device pool and $6.5\times$ the host tier, so at most 2.9 sessions can be parked at once and all the others must lose their state. The decision that matters is a replacement decision between sessions.

Its marginal condition follows from (2): holding s rather than t is worth it when the prefill time kept alive per byte is larger,

$$\frac{p_s\, T_{\text{rec}}(s)}{\Phi_s} \;>\; \frac{p_t\, T_{\text{rec}}(t)}{\Phi_t}, \tag{6}$$

with p_s the probability that the session returns before its state would be evicted anyway. The controller instantiates (6) in two coarse but observable forms: victims are the youngest (smallest invested context, so smallest T_{rec}), and a session is suspended only after it has been idle longer than the time it takes to run a replacement round,

$$t^* \;=\; \text{time to serve one round of another session}, \tag{7}$$

because a session that returns sooner than t^* would pay a revival for a slot the replacement could not have used. In the long-session trace, $t^* = [PENDING : t - star]$.

Equation (6) is a *replacement* condition, not a per-session one: when the host tier is full, suspending a session always ejects another, and the cost of a suspension is the revival-or-recompute it causes elsewhere. This is why the design tracks host occupancy instead of treating the host tier as free capacity.

C. Prop. 3: a wrong growth reserve cannot move the ledger far

Write the projection error of session s as $\Phi_s^{\text{next}} - \hat{\Phi}_s = g_s - \hat{g}_s$, with g_s the realised growth. Then

$$\frac{|\Phi_s^{\text{next}} - \hat{\Phi}_s|}{\Phi_s} \;\le\; \frac{g_s}{\Phi_s} + \frac{|\hat{g}_s - g_s|}{\Phi_s} \;\le\; \varepsilon_s + \varepsilon'_s, \qquad (8)$$

where $\varepsilon_s = g_s/\Phi_s$ is the growth-to-footprint ratio and ε'_s its estimation error, both normalised by the same footprint. On the trace the reserve $\hat{g}_s$ has a median of 3.7K tokens against a 48.5K-token context, so $\varepsilon_s \approx 7.7\%$, and it covers the realised growth in 99.5% of the 583 round transitions. A reserve that is wrong by a factor of two therefore moves the ledger by less than a tenth of a footprint, which is smaller than the spread between the mean session (69.0K) and the median one (48.5K). The projection is dominated by measured bytes, not by the estimate.

Two readings follow. First, the design does not need an accurate predictor, because the predicted term is a few percent of the quantity it protects—an argument that would not hold in a regime where a round appends a large fraction of its context. Second, the reserve is what keeps admission *conservative* rather than what makes it accurate: it absorbs the case where a session's next round is unusually large, and its cost is proportional to its size, which the trace bounds at ε_s.

We report the empirical version of (8) in §VII: the distribution of the realised projection error over all admissions, and the fraction of admitted sessions whose next round exceeded the reserve.

VI. Implementation

a) Engine: We use SGLang v0.5.10 [2] *unmodified*, with paged KV [3] and its hierarchical cache enabled at a host ratio of 2× with write-through policy. The model is Qwen2.5-Coder-7B-Instruct with YaRN extrapolation to a 96K context, whose weights and KV cache fit a single 24 GB RTX 3090 at a static memory fraction of 0.85, leaving a device pool of 101,502 tokens and a host tier of 203,004. The engine exposes token usage, running-request counts, and cache statistics on its metrics endpoint; it has no concept of a session and no notion of suspension. Nothing in the engine was changed to support this paper—the design lives entirely outside it.

b) Harness: A session runner drives the replay: for each session it emits an arrival event, then for each round it waits for a permit, submits the round to the engine, records the step (prompt, cached and uncached tokens, first-token latency, duration), and simulates the inter-round tool wait before emitting the next round's arrival. The runner implements the round-boundary gate: a session that is not in the admit set blocks before its next request is issued, keeping its process state and its engine-side KV. The runner is written in Rust and is the only component that talks to the engine.

c) Controller: The controller is a Python process of 540 lines that polls the engine's metrics and the runner's event logs every two seconds, maintains the ledger of §IV-A, evaluates the valve and the shedding rules of §IV-D, and rewrites the permit file atomically with full-state semantics: each cycle publishes the complete admit set and the complete paused set, so the runner never acts on a stale delta. Decision records—resident bytes, projection, budget, valve state, running count, shed set—are appended to a log that the evaluation reads. The controller never opens the trace file: the only quantities it sees are the ones the deployment produces at run time.

d) Arms: All three arms use the same engine, model, trace, memory fraction, and cache configuration; they differ only in the control process, which is selected by a policy string. The *request-level* arm runs with no controller at all—the engine admits every round on arrival and its own scheduler arbitrates. The *agent-level* arm runs a Concur-style AIMD controller [4] that adjusts a concurrency window from observed utilisation and a cache-thrash signal, pausing and resuming sessions at step boundaries with the same named-permit mechanism. The *capacity-accounted* arm is the design of §IV. Using one permit mechanism for both controllers matters: the comparison then isolates the decision rule (AIMD on a window versus accounting on a ledger), not the engineering of pausing.

e) Reproducibility: A run is one command per arm and writes a directory containing the per-step log, the controller's decisions, the admission events and a metadata file with the code hashes, trace hash, GPU, and pool size. The evaluation scripts in the artifact recompute every number in §VII from those directories.

VII. Evaluation

Our evaluation asks three questions. Does accounting beat the two baselines that a practitioner would use instead (§VII-B)? Where does the time go—is the win an admission effect or a cache effect (§VII-C)? And when does it stop paying (§VII-G)? We fix the metrics and the decision criteria before running, and report every arm on every metric whether or not it favours the design.

A. Setup and metrics

All arms replay the same 19-session, 602-round trace on the same GPU, model, memory fraction, and cache configuration (§VI); they differ only in the controller. We report five quantities, defined once and computed by one script from the per-step logs.

Makespan: the time from the first arrival to the last completion—the batch time a server operator cares about. *Session end-to-end*: for each session, the time from its arrival to its own last completion; we report the median and the 90th percentile, because the mean hides the tail. *Round latency*: for each round, the time from the moment the session *could* continue (its arrival, or the end of the previous round plus the simulated tool wait) to the first output token. This is the quantity a user experiences, and it includes both the time the controller makes the session wait at the door and the time the engine takes to produce a token; reporting engine-side TTFT alone would hide exactly the cost that admission control imposes. *Deadline miss rate*: the fraction of rounds whose

round latency exceeds 10 s, the per-round budget we fixed in advance. *Goodput*: rounds that meet the deadline per hour of makespan, which normalises for the fact that a faster batch also has less wall time in which to accumulate hits. *Work*: the sum of uncached prompt tokens and completion tokens, i.e. the prefill the engine actually had to do, which is the quantity the mechanism claims to reduce.

B. Main result

Table II reports the three arms.

[PENDING: two paragraphs: the headline comparison (makespan and goodput), and the honest reading of the parts that do not separate. State explicitly which of the four pre-registered criteria hold.]

C. Where the time goes

A faster makespan can come from three places: the engine does less work (fewer uncached prefill tokens), the engine is busier (less idle time while sessions are parked), or the work is better overlapped. We separate them from the logs, without running anything new. The re-read tax is the ratio of uncached prefill to appended tokens: it is 1 when every session's prefix is retained and grows when state is lost.

[PENDING: paragraph interpreting the decomposition, including the cases where the design wins on work but not on busy time, or vice versa.]

D. Where the recompute comes from

Table III says how much work was redone; it does not say who caused it. We classify every round of an arm by what happened between the previous round's completion and this round's submission: the round is the session's *first* (nothing to reuse), it followed a *suspension* by the controller, or it was *continuous* (admitted throughout, so a miss is the engine's own eviction). The classification is mechanical—it reads the admission event log and the step log, and needs no new run.

[PENDING: paragraph: the reading. In the regime we study the host tier holds only a few sessions, so a suspension often cannot keep the state it suspends; the table quantifies that, and it is the empirical form of the claim that the slot, not the copy, is the scarce resource. State plainly what fraction of the recompute is the design's own doing.]

E. Mechanism, in one session

[PENDING: paragraph reading the timeline: how often the session was suspended, what the suspension cost (revival vs recompute counts), and whether the slot was picked up by another session while it was parked.]

F. Log-based counterfactual: concurrency or state?

Our design differs from the agent-level baseline in two ways at once: it accounts for capacity, and it preserves state. To separate the two without another run, we replay the controller's own decisions offline and ask what would have happened under a concurrency-only variant of the same controller—same permits, same pausing, but no host-tier accounting and no revival pricing.

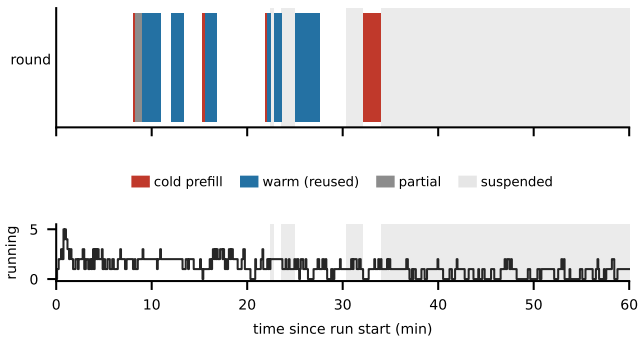


Fig. 2: One session of the capacity-accounted arm (39 rounds, 17 suspensions), first hour. Each bar is a round, coloured by how much of its prefix the engine had to recompute; grey spans are suspensions. The lower panel is the engine's running-request count: while the session is suspended the card keeps serving other sessions, which is the slot the suspension buys. A round that follows a suspension is usually red—the state did not survive—which is the cost side of the same decision.

[PENDING: the counterfactual numbers and the sentence that says how much of the gain comes from each half.]

G. When it does not pay

We report three regimes in which the mechanism loses or is neutral, and we say why. *(i) Ample capacity.* If the working set fits, nothing is suspended, the ledger reduces to first-come-first-served, and the only difference from the request-level arm is a small amount of bookkeeping; the design cannot win and does not claim to. *(ii) Short contexts.* If a round's context is small, the recompute is cheap and the break-even (5) favours recomputation; suspension then buys nothing. *(iii) Saturated engine.* When the engine is already busy, adding admission discipline removes capacity from the queue and can increase the tail, because the door wait moves from a place where it overlaps with other work to a place where it does not.

[PENDING: the measured instances of (i)–(iii) in the arms, and the sensitivity to ρ and t^*.]

H. Threats to validity

Every arm is a single run of one trace on one GPU, so we report measured differences and full distributions rather than significance tests; run-to-run variation is visible only in the per-session scatter, and a single batch cannot separate policy effects from the particular interleaving. The trace is reconstructed from agent transcripts rather than observed in production, and all sessions arrive together, which is the worst case for admission and the best case for a batch controller. The model is a 7B coding model on a 24 GB card; the constants in §V would differ on other hardware, though the quantities in which the argument is stated would not.

VIII. Related Work

a) KV memory in serving engines: PagedAttention [3] made the KV cache a paged allocator, and SGLang [2] made

TABLE II: Three arms, same trace, engine, and memory budget. Round latency includes the controller's door wait and the engine's TTFT; the deadline is 10 s per round. Work is uncached prefill plus completion tokens.

	request-level	agent-level	FOYER
makespan (min)	[PENDING: ms-A]	[PENDING: ms-B]	154.9
session end-to-end, median (min)	[PENDING: e2e-A]	[PENDING: e2e-B]	115.6
session end-to-end, p90 (min)	[PENDING: e2e90-A]	[PENDING: e2e90-B]	151.8
round latency, median (s)	[PENDING: r50-A]	[PENDING: r50-B]	9.7
round latency, p90 (s)	[PENDING: r90-A]	[PENDING: r90-B]	291.9
rounds missing 10 s (%)	[PENDING: miss-A]	[PENDING: miss-B]	49.8
goodput (rounds/h)	[PENDING: gp-A]	[PENDING: gp-B]	105.4
work (Mtok)	[PENDING: work-A]	[PENDING: work-B]	9.00
KV reuse (%)	[PENDING: hit-A]	[PENDING: hit-B]	65.2

TABLE III: Decomposition. Busy and idle time are sampled from the engine's running-request count over each run's window. Prompt, device-hit and host-hit tokens are the engine's cumulative counters over the same window; uncached prefill, completion and the re-read tax are summed from the per-step records. Evicted tokens are what the engine had to throw away to make room.

	request-level	agent-level	FOYER
busy time (min)	[PENDING: busy-A]	[PENDING: busy-B]	107.5
idle time (min)	[PENDING: idle-A]	[PENDING: idle-B]	51.1
prompt tokens (Mtok)	[PENDING: pr-A]	[PENDING: pr-B]	25.23
uncached prefill (Mtok)	[PENDING: unc-A]	[PENDING: unc-B]	8.77
completion (Mtok)	[PENDING: dec-A]	[PENDING: dec-B]	0.23
device hits (%)	[PENDING: dh-A]	[PENDING: dh-B]	43.7
host hits (%)	[PENDING: hh-A]	[PENDING: hh-B]	21.5
evicted tokens (Mtok)	[PENDING: ev-A]	[PENDING: ev-B]	15.32
re-read tax (× append)	[PENDING: rr-A]	[PENDING: rr-B]	4.43

TABLE IV: Attribution of the re-read tax. Rounds are classified by what happened between the previous round and this one. *rounds* and *share* are percentages, *uncached* is in millions of tokens, and *excess* is uncached prefill divided by the tokens the round actually appended.

	first	susp.	cont.
rounds	3.5	24.9	71.6
uncached (M)	0.33	4.80	3.65
share	3.7	54.7	41.6
excess	1.75	6.99	3.29

prefix sharing a first-class citizen through a radix tree; both are the substrate we stand on and neither has a notion of a session. Hierarchical and disaggregated stores extend the hierarchy—host memory, remote tiers, and prefill/decode separation [5], [6]—and answer *where* a block should live. Our question is prior to theirs: given that the hierarchy exists, which session is allowed to occupy the fast tier, and what does it cost to bring one back. Pensieve [7] argued that statelessness is the structural fault of current serving designs; we make the same argument quantitatively and add the admission decision that statelessness leaves open.

b) Scheduling for agentic and multi-turn workloads: CONCUR [4] controls concurrency at agent granularity with an AIMD law driven by a congestion signal, and it is the closest system to ours in scope; our agent-level arm is deliberately a faithful reimplementation of that control law, so that the comparison isolates the decision rule. We differ in what is being decided: AIMD adjusts a window and reacts after congestion appears, whereas the ledger tests feasibility before admitting and prices what happens when it cannot. Continuum [8] attaches a time-to-live to a session's KV and uses cache miss cost to schedule, which is a prediction of the reuse window; our reserve is a measurement of growth, not of a horizon. KVFlow [9] predicts steps-to-execution to schedule workflows, and ThunderAgent [10] decomposes program cost in space and time; both model the future of a program, and both would benefit from a feasibility layer underneath. Talaria [11] targets session continuity in serverless serving, and ConServe changes the scheduling unit; what the unit is allowed to own. MORI [12] exploits tool-call idle windows to offload per tier, and EfficientAgent admits by working set for eviction by expected reuse; our problem statement overlaps theirs, and we make no claim of novelty on granularity, tiers, or working-set admission as such. The claim here is that the decision can be stated as a feasibility test with a measured ledger and a priced recovery, without a predictor of future demand.

c) Measurement of agentic workloads: A recent characterisation of production agentic traffic reports at a scale where the aggregate working set would need hundreds of terabytes to hold; the conclusion there is that hold-all-live is impossible, which is the regime we make precise on one card: a working set 12.9× the pool, a pool that holds 1.47 mean sessions, and a 13× re-read tax paid by request-level serving. Our trace is

TABLE V: The regime the design targets, and where it does not apply.

Applies	Does not apply
working set $\gg$ KV pool: long sessions, large contexts	working set well below the pool: nothing is suspended and the ledger is inert
single node with a host tier that can hold suspended state	cluster-level disaggregation, where placement is a different problem
contexts large enough that recomputation dominates a copy	contexts short enough that the recompute is cheaper than the revival
sessions that return: multi-round jobs with reusable prefixes	one-shot requests, where there is no state to preserve

a replay built from coding-agent transcripts [1], so it inherits their structure rather than observing a deployment; we state that as a threat to validity rather than as evidence of production behaviour.

d) Admission and overload control: Classical admission control reacts to utilisation with multiplicative decrease [13]; the working-set model [14] and Little's law [15] describe when a set of jobs fits in memory and what concurrency a throughput target implies. We use the same vocabulary but a different object: the working set here is a set of *sessions* with reusable state, so the cost of admitting the wrong one is not queueing delay but recomputation, and the relevant invariant is a byte budget rather than a window. Overload handlers that reject requests [5] are the limiting case of our design in which every suspension is made permanent; our point is that a rejection can be made reversible, and that the break-even (§V) says when it should not be.

IX. Discussion

a) What the argument is, and is not: The claim is not that this card, this model, or this trace is representative. It is that once sessions can be suspended, admission stops being a forecast and becomes a feasibility test with a price, and that this reframing is what turns an unmanaged pool into a usable server under working-set overcommitment. The quantities in which the argument is stated—bytes of KV, bytes per second between tiers, rounds per session, rounds per second of prefill—are measurable on any deployment; the constants are not portable, and we do not present them as such.

b) Why the negative cases matter: A design of this kind is easy to oversell, because every mechanism in it can be made to look good on the workload it was tuned for. The three losing regimes in §VII-G are therefore part of the result: they delineate the ratio of working set to capacity at which accounting stops being the interesting question, and they explain why threshold controllers are not simply mistakes—in a regime where the working set is small, a window is a perfectly good answer, and the ledger has nothing to add.

c) Implications for systems that already have a hierarchy: The mechanism does not require a new engine. It requires that the engine expose enough for a controller to keep books—token usage, running counts, per-request cache accounting—and that the scheduler allow a request to be held at a session boundary rather than only admitted or dropped. Both are interface changes, not algorithmic ones, and both are absent from current engines mainly because the request, not the session, is their unit of work.

d) Threats beyond the evaluation: Our replay arrives all at once, which makes admission hardest and makes a batch controller look good; a Poisson arrival process would give the request-level arm more slack and shrink the measured gap, though it would not change which decisions are feasible. The trace's tool waits are simulated from the recorded distribution rather than observed live, so the interaction between admission delays and agent behaviour (an agent that waits longer may emit a longer next round) is not modelled. Both are directions for measurement rather than for mechanism.

X. Conclusion

Serving agents is not serving requests. A session arrives with state, leaves it behind, and comes back to it, and on a single GPU the aggregate of that state exceeds the pool by an order of magnitude. The systems we have are built around a threshold—how many, how much concurrency, how high a watermark—which forces the operator to guess at a future the workload does not reveal until it happens.

Once a session can be suspended and revived, the guess is unnecessary. Capacity becomes a ledger of bytes, admission becomes a feasibility test against that ledger, and the two decisions that remain—whose state deserves a slot, and whether to copy or recompute—are priced in measured time. The design needs no predictor of the next round, and the numbers say why: the part of the footprint that would have to be predicted is a few percent of the part that is measured.

We have instantiated the principle as a controller outside an unmodified serving engine and measured it against request-level serving and an agent-level feedback controller on a 19-session coding-agent replay. [PENDING: one-sentence result with the three arms]. The reusable part is not the constants but the shape of the argument: state the feasible set, price the recovery, and let the workload decide—rather than tune a threshold and hope the next round looks like the last.

References

[1] K. Zhu, M. Jacob, C. Ma, Y. Pan, S. Wang, A. Krishnamurthy, and B. Kasikci, "TraceLab: Characterizing coding agent workloads for LLM serving," 2026, preprint; this paper's trace source.

[2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 62 557–62 583, arXiv:2312.07104; v1 title was "Efficiently Programming Large Language Models using SGLang".

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 611–626.

[4] Q. Chen, Z. Ye, T. Tang, P. Sun, B. Tian, G. Wang, S. Li, Y. Wen, Z. Han, and T. Zhang, "CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control," in *International Conference on Machine Learning (ICML 2026)*, 2026, poster, Seoul, July 2026; OpenReview id U7c6glPdDW; arXiv:2601.22705.

[5] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *ACM Transactions on Storage*, vol. 22, no. 4, pp. 1–38, 2026.

[6] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 193–210.

[7] L. Yu, J. Lin, and J. Li, "Stateful large language model serving with Pensieve," in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025, pp. 144–158.

[8] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," 2025, preprint, no venue.

[9] Z. Pan, A. Patel *et al.*, "KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows," 2025, preprint; author list and title need verification before submission.

[10] H. Kang, Z. Li, W. Xu, X. Yang, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, "ThunderAgent: A simple, fast and program-aware agentic inference system," 2026, preprint, no venue.

[11] U. Meng, U. Zhao, D. Li, G. Gao, and F. Du, "Talaria: Session-aware serverless serving of hundred-billion-parameter LLMs," 2026, preprint; author names as listed on arXiv.

[12] T. Xia, H. Li, Z. Li, X. Chen, H. Kang, Y. Qiao, Y. Xu, and I. Stoica, "Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI," 2026, preprint, no venue.

[13] V. Jacobson, "Congestion avoidance and control," in *Symposium Proceedings on Communications Architectures and Protocols (SIGCOMM '88)*, 1988, pp. 314–329.

[14] P. J. Denning and J. P. Buzen, "The operational analysis of queueing network models," *ACM Computing Surveys*, vol. 10, no. 3, pp. 225–261, 1978.

[15] J. D. C. Little, "A proof for the queuing formula: $l = \lambda w$," *Operations Research*, vol. 9, no. 3, pp. 383–387, 1961.